Anforderungen, Akzeptanztests und Betrieb in einer Plattform.

KnippQAi verbindet die Beschreibung eines Systems mit seinen ausführbaren Akzeptanztests und dem Blick in den Betrieb. Fachbereich und Entwicklung arbeiten am selben Stand, eine KI unterstützt jeden Schritt, und alles liegt in Standardformaten im Git-Repository des Kunden.

Story › Refinement › Ready › Entwicklung › Abnahme › Release › Betrieb

ANFORDERUNGEN

Systemmodell

- Vision, Ziele, Stakeholder, Use Cases, Geschäftsregeln, Entitäten, Masken, API-Operationen und Qualitätsanforderungen nach ISO 25010
- Formulare je Element, Vorschau als Diagramm, Maske oder Schnittstellendoku
- Verweise in beide Richtungen, Modellansicht mit Lücken
- Ablage als AIUP-Markdown, OpenAPI 3.1, Mermaid, C4 und PlantUML
- Bestehende Repositories anbinden und einlesen

ANFORDERUNGEN

Stories und Refinement

- Story als eigener Zweig über das ganze Projekt, mit Epic und Sprint
- Umfang wird aktiv aufgenommen, Vergleich mit main Feld für Feld
- main ist gesperrt, jede Änderung beginnt in einer Story
- KI-Rückfragen im Refinement, Antworten werden Änderungsvorschläge
- Ready-Check mit Umsetzungsprobe, erneute Prüfung nur der Änderungen
- Verlauf und Zurücksetzen über Git

AKZEPTANZTESTS

Editor und Step-Katalog

- Gauge und Cucumber, Step-Code in TypeScript oder Java
- Geführter Editor für den Fachbereich, Textansicht für die Entwicklung, dieselbe Datei
- Tabellen, Concepts, Parameter, Tags, Testdaten
- Autovervollständigung aus dem Katalog, Verwendung über alle Zweige
- Fehlende Implementierungen per Assistent, einzeln oder gesammelt
- Testebene UI, API oder Messaging folgt aus dem Step-Code

AKZEPTANZTESTS

Live-Sitzung

- Anwendung und Szenario nebeneinander, Schritt für Schritt
- Element auf der Seite wählen, die KI formuliert den Satz
- Nachweise aus Browser, REST-Aufrufen und Kafka mit Avro
- Wiederholte Aufrufe zu einer Zeile gefaltet
- Probieren, übernehmen oder verwerfen, bevor etwas in der Spec landet

KI

Assistent mit Werkzeugen

- Arbeitet über MCP mit den Werkzeugen des Projekts
- Assistiert oder autonom, Freigabe je Aktion oder Sitzung
- Änderungen als Diff, auch über mehrere Dateien
- KI am Element: vervollständigen, umschreiben, Kürzel erklären
- Kosten je Story und Prüfung sichtbar, Anbieter konfigurierbar
- MCP-Server für Coding-Agenten in der IDE

AUSFÜHRUNG

Läufe und Nachweis

- Läufe in Gauge-Environments auf eigenem Runner je Mandant
- Fachliche und technische Sicht, Schritte mit Zeiten und Vorgängen
- Gauge-Report, PDF-Bericht und JUnit-XML
- Bruchanalyse mit KI-Erklärung
- Instabile Tests als eigene Kennzahl

ABNAHME

Review und Rückführung

- Board von Akzeptanz vereinbart bis Akzeptiert
- Abnahmetor: neue Szenarien grün, nichts schlechter als auf main
- Vier-Augen-Prinzip mit Rolle Abnehmer
- Tore mit Begründung übersteuerbar
- Rückführung als Pull Request ins Repository des Kunden

REGRESSION

Testgruppen und Release

- Testgruppen als gespeicherte Tag-Abfrage
- Zeitpläne in Worten statt Cron-Ausdruck
- Verlauf: neu rot, wieder grün, unverändert
- Release bündelt fertige Stories je Domäne
- Freigabe je Umgebung, Regression vor dem Release

BETRIEB

Lage und Wirkung

- Lage als Tagesbilanz mit Zeitstrahl
- Was frisch live ist, neue Fehler je Operation mit Bezug zur Story
- Messpunkte und Qualitätsziele gelten ab dem Merge
- Absprung nach Grafana, Loki und Tempo

PLATTFORM

Mandanten und Rollen

Mandanten aus Keycloak, Projektrollen Lesen, Autor, Abnehmer und Admin.

Administration

Betreiber- und Mandantenebene, Abrechnung nach Token, Läufen und Sitzen.

Git-Anbindung

GitHub, GitLab und Gitea, auch selbst betrieben. Kein Exportformat, kein Lock-in.

Betrieb

Kubernetes oder Docker Compose, Runner je Mandant isoliert.

STANDARDS Gauge · Cucumber · TypeScript · Java · Playwright · OpenAPI 3.1 · Kafka · Avro · AIUP · OpenSpec · Mermaid · C4 · PlantUML · MCP · Keycloak · Grafana · Kubernetes